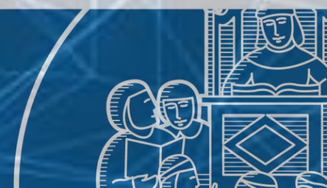
ROBOT PROGRAMMING FRAMEWORKS AND IOT PLATFORMS

Scuola Superiore Sant'Anna

Anno Accademico 2025/2026

Gastone Ciuti

IMT SCUOLA ALTI STUDI LUCCA



Organization

11/11/2025 Introduction and Communication Protocols (1/2) (3 hours – SLOT 1)

17/11/2025 Communication Protocols (2/2) (3 hours – SLOT 1)

18/11/2025 IoT Platforms and Main Features (3 hours – SLOT 2)

24/11/2025 Set-up of Particle Development Boards & Basic Functions (2 hours)

- Web-based programming, cloud functions and examples

25/11/2025 How to Program an IoT System: Functions and Coding (1/3) (3 hours – SLOT 3)

- Other Programming Environments, *i.e.* Particle Workbench based on Visual Studio Code

01/12/2025 How to Program an IoT System: Functions and Coding (2/3) (2 hours – SLOT 3)

- Advanced Programming Functionalities for IoT Platforms and Use Cases

02/12/2025 How to Program an IoT System: Functions and Coding (3/3) (3 hours – SLOT 3)

- Integration with Web Services, *i.e.* IFTTT (if this than that)

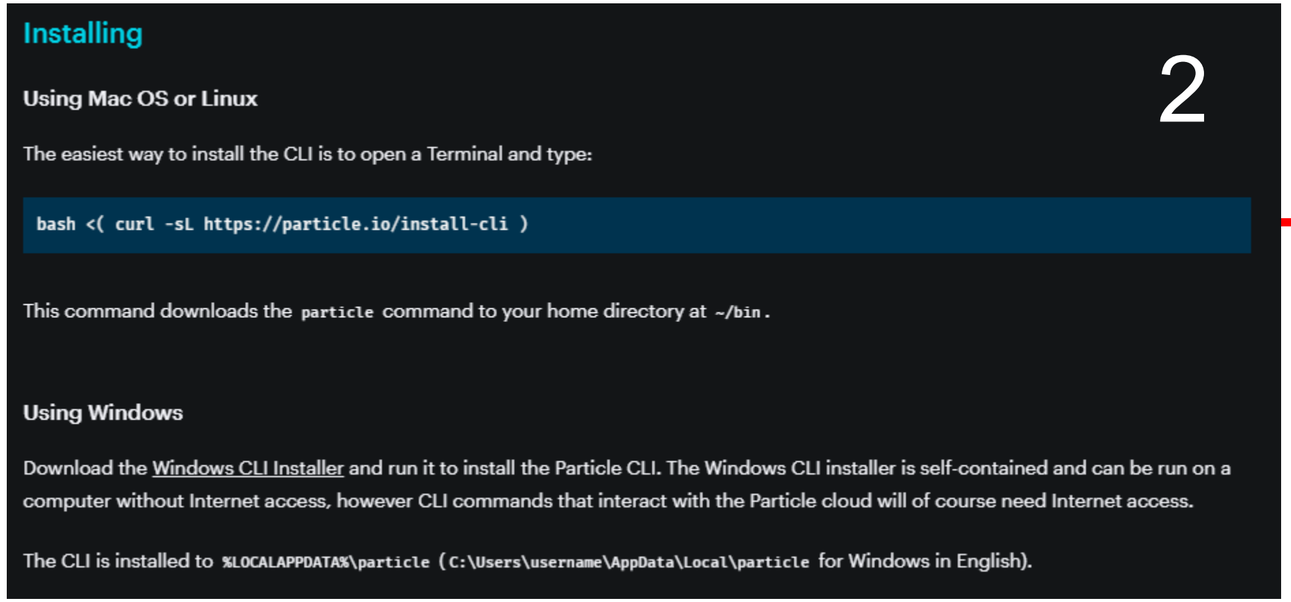
09/12/2025 Examples with IoT Systems and Exercises (3 hours – SLOT 4)

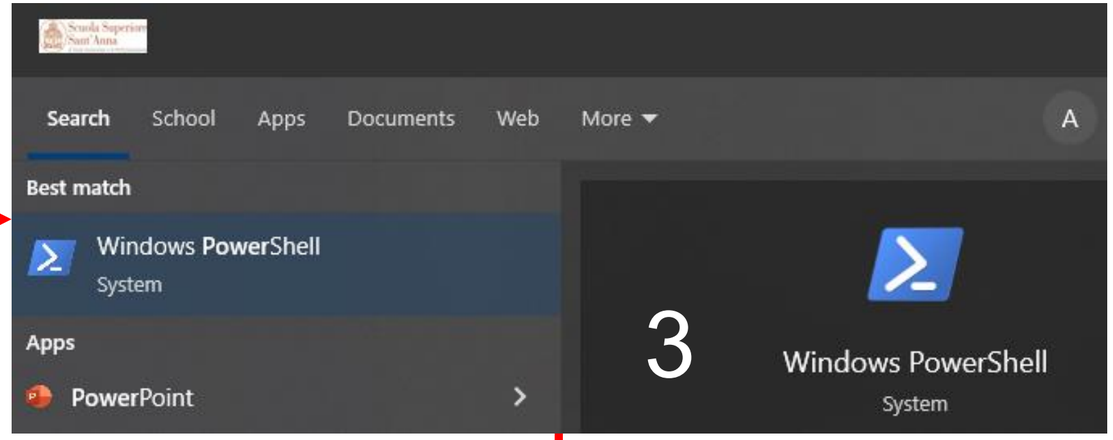
Set-up of Particle Development Boards & Basic Functions (02.12.2025) - SLOT 3b

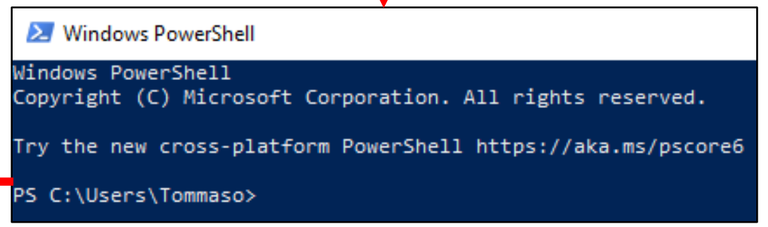
Gastone Ciuti

Installation of Particle CLI

1. Download the Package for installation: LINK
2. Follow the steps for installation HERE
3. Open the PowerShell (windows) or the Terminal (Linux, MacOS)







4. Type “particle” and should print all the available keys of the CLI

Particle account Login

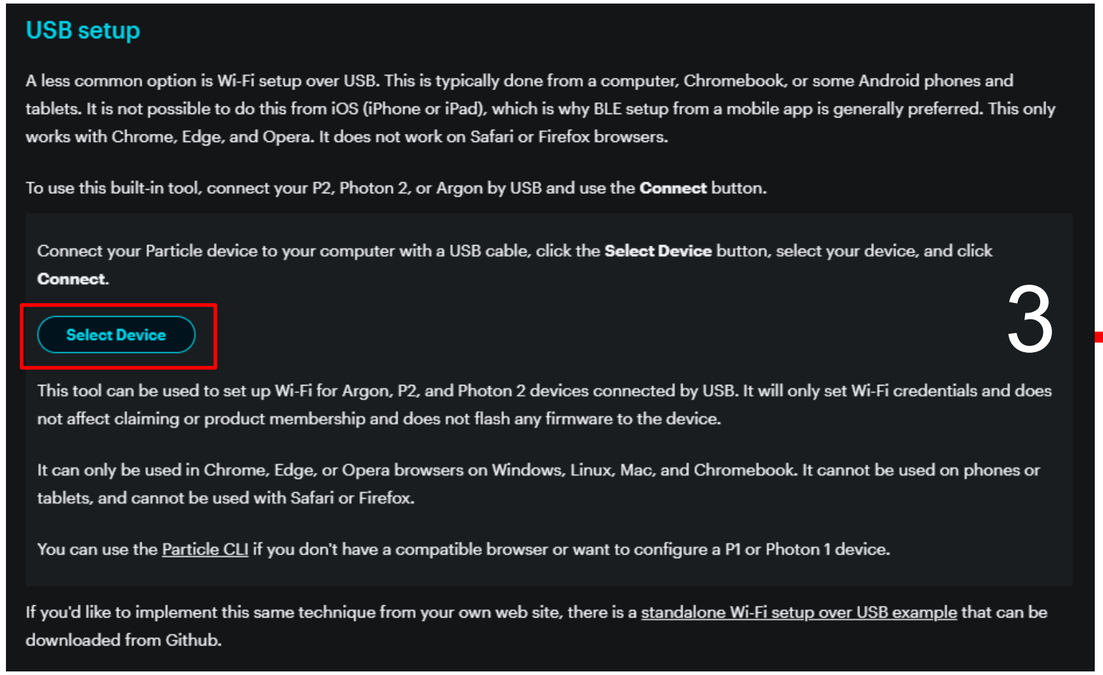
Open the PowerShell (windows) or the Terminal (Linux, MacOS)

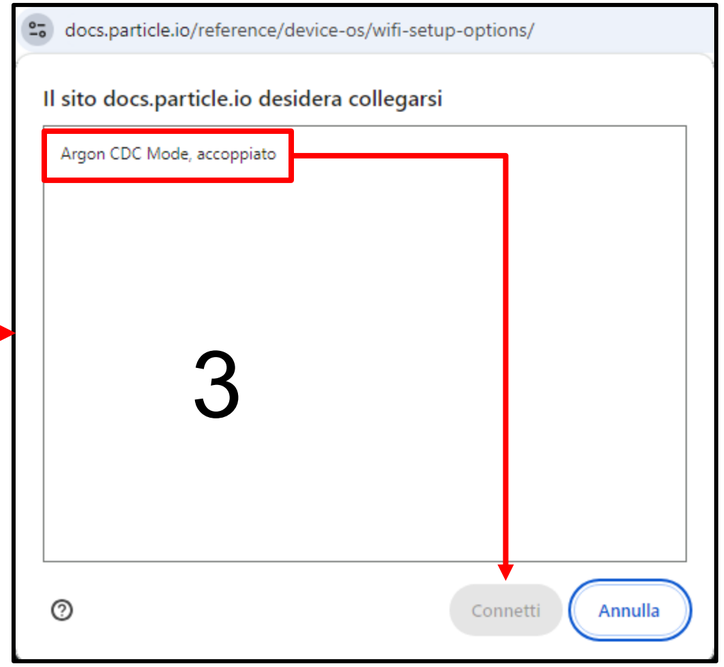
`particle whoami` (shows the logged username)
`particle login` (type the credential to login)

```
⊗ angelo@Jarvis Particle $ particle whoami
  You're not logged in. Please login using particle login before using this command
○ angelo@Jarvis Particle $
○ angelo@Jarvis Particle $
○ angelo@Jarvis Particle $
○ angelo@Jarvis Particle $
● angelo@Jarvis Particle $ particle login
  ? Please enter your email address bionics20212022@gmail.com
  ? Please enter your password [hidden]
  > Successfully completed login!
```

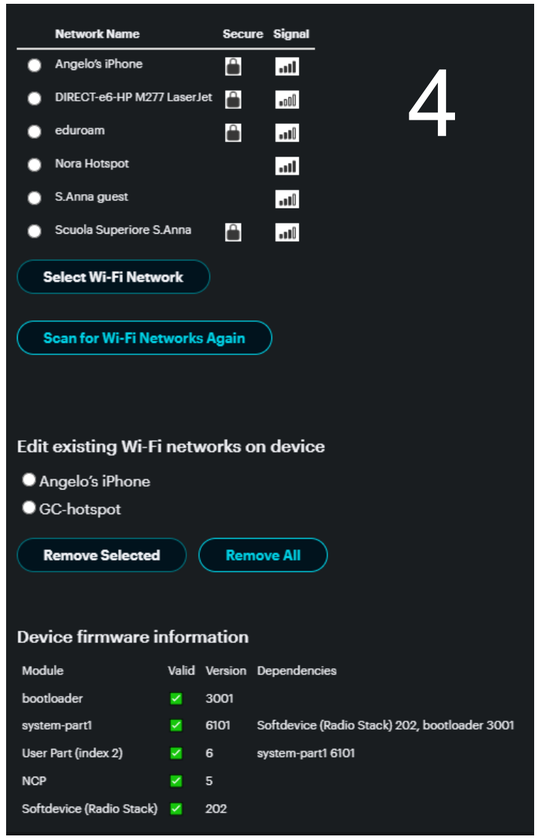
Forcing wi-fi connection (Web-site)

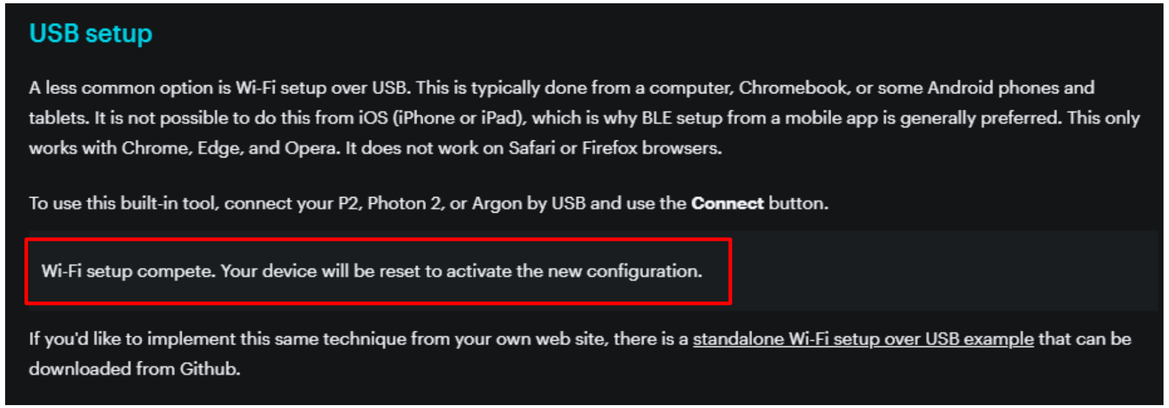
1. Wi-Fi Setup Page: LINK
2. Connect Particle to USB port of your PC
3. Scroll the webpage until the end





4. Choose the network
5. Force connection with your hotspot





Forcing wi-fi connection with CLI

1. Make sure the device is connected to the usb cable.
2. Make sure the device is blinking green (looking for wi-fi).
3. Open the PowerShell (Windows) or the Terminal (Linux, MacOS)
4. Type the following commands:

particle wifi join (choose the network and connect)
particle wifi current (Verify your connection)

```
angelo@Jarvis src $
angelo@Jarvis src $ particle wifi join
? Would you like to scan for Wi-Fi networks? Yes
? Select the Wi-Fi network with which you wish to connect your device: Angelo's iPhone
? Wi-Fi Password Angelo123
Wi-Fi network 'Angelo's iPhone' configured successfully. Attempting to join...
Use particle wifi current to check the current network.
angelo@Jarvis src $
```

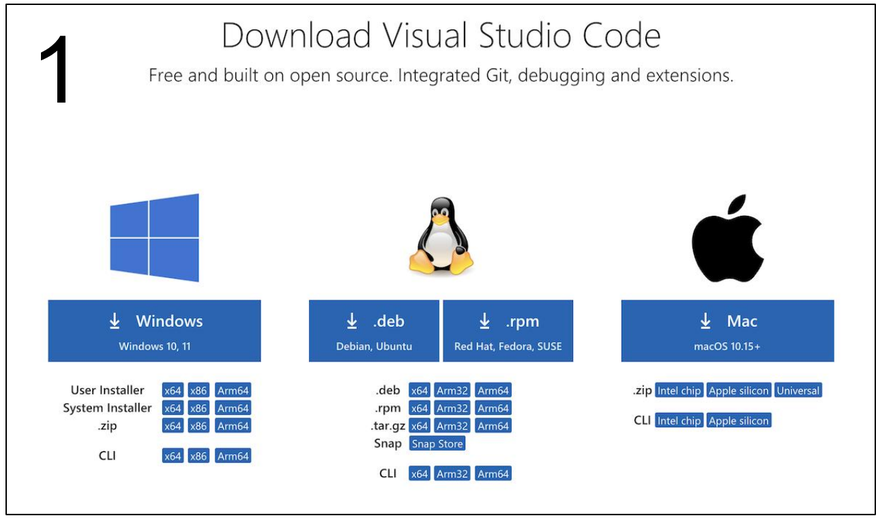
5

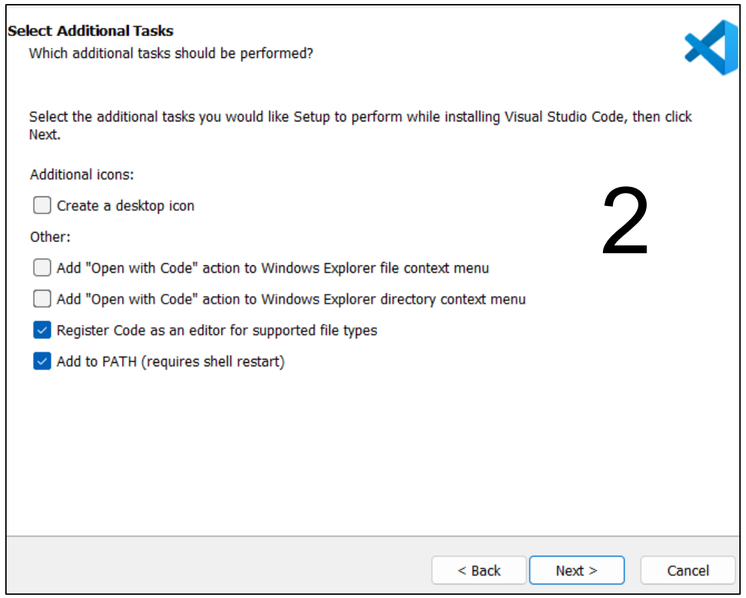
```
angelo@Jarvis src $ particle wifi current
Current Wi-Fi network:

- SSID: Angelo's iPhone
  BSSID: 1e:3c:62:70:93:9a
  Channel: 6
  RSSI: -33

angelo@Jarvis src $
```

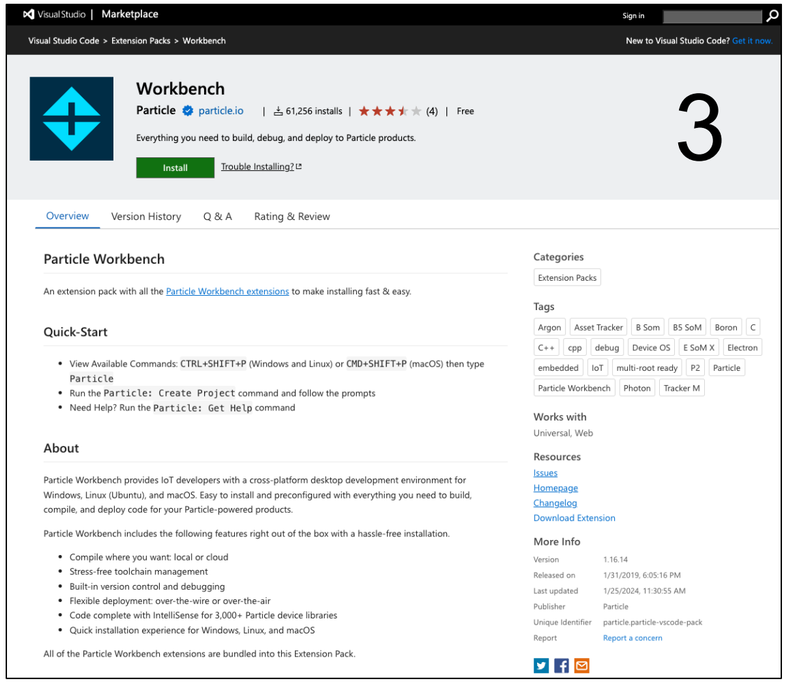
6

Installation of Visual Studio Code

1. Visual Studio Download: LINK
2. VS installation guide here: LINK
3. VS Market place: LINK

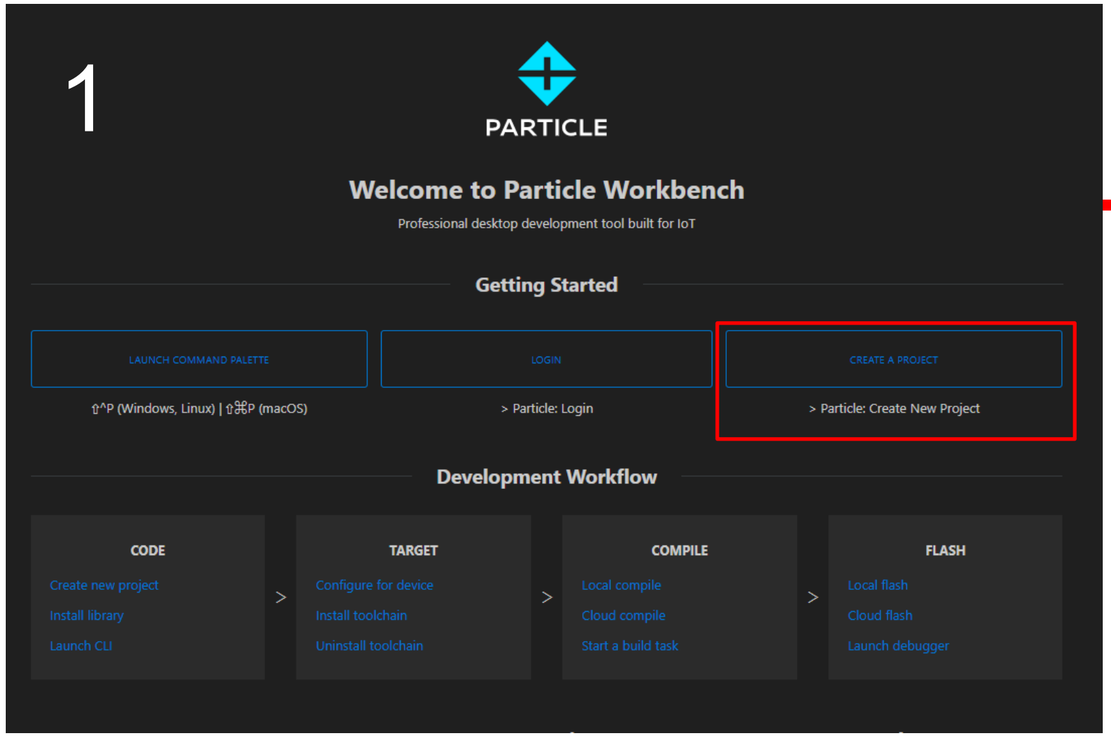


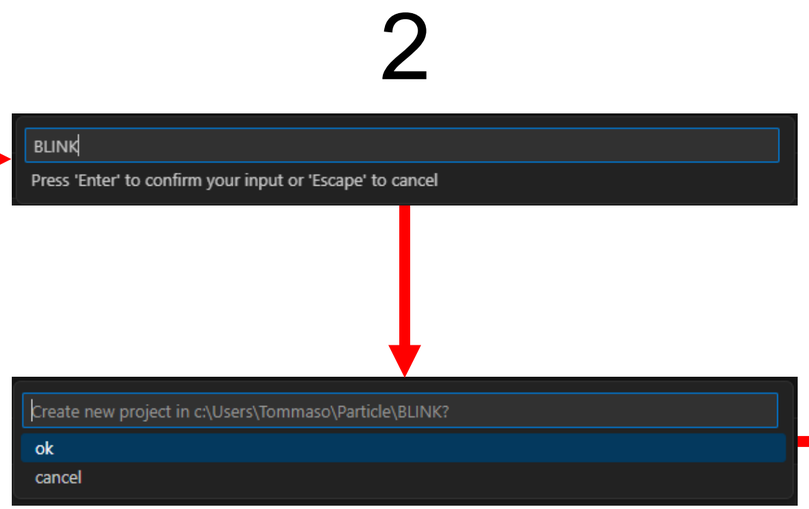


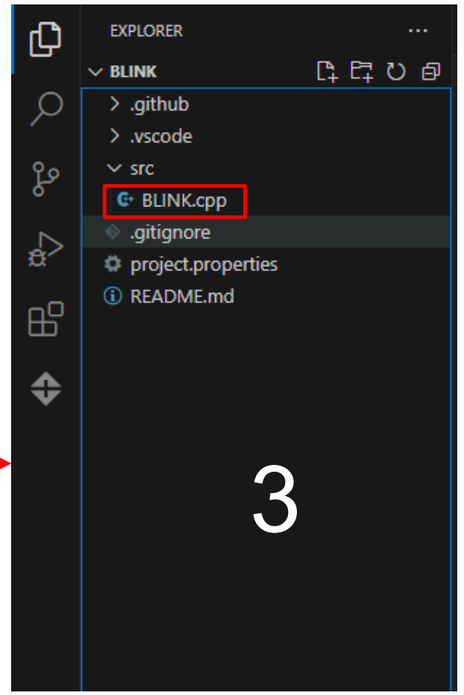


Compiling and Flashing the Code

1. Create a Project in VS and choose the folder
2. Give a name to your project
3. The code is created automatically

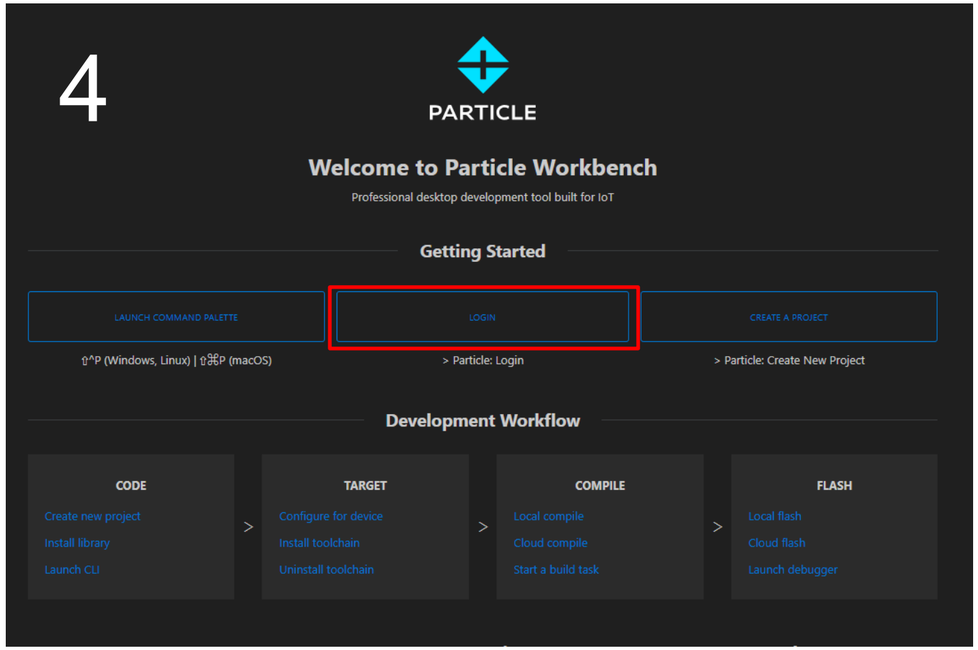




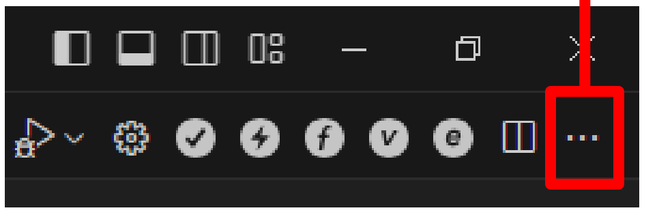


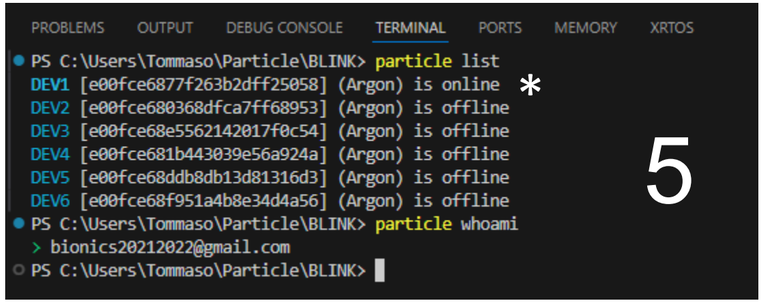
4. Login to the Particle account
5. Check if the cloud is connected
6. Select the device to flash the code

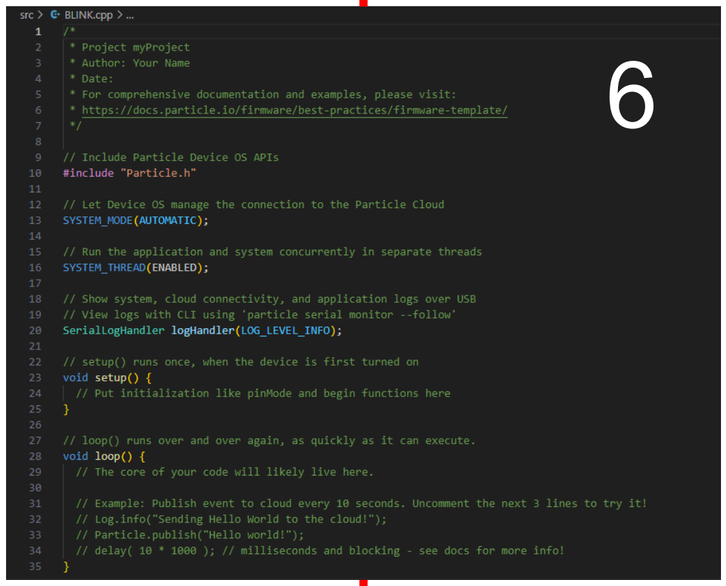
Code generated automatically by VS

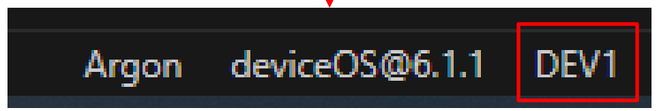


Launching Particle CLI



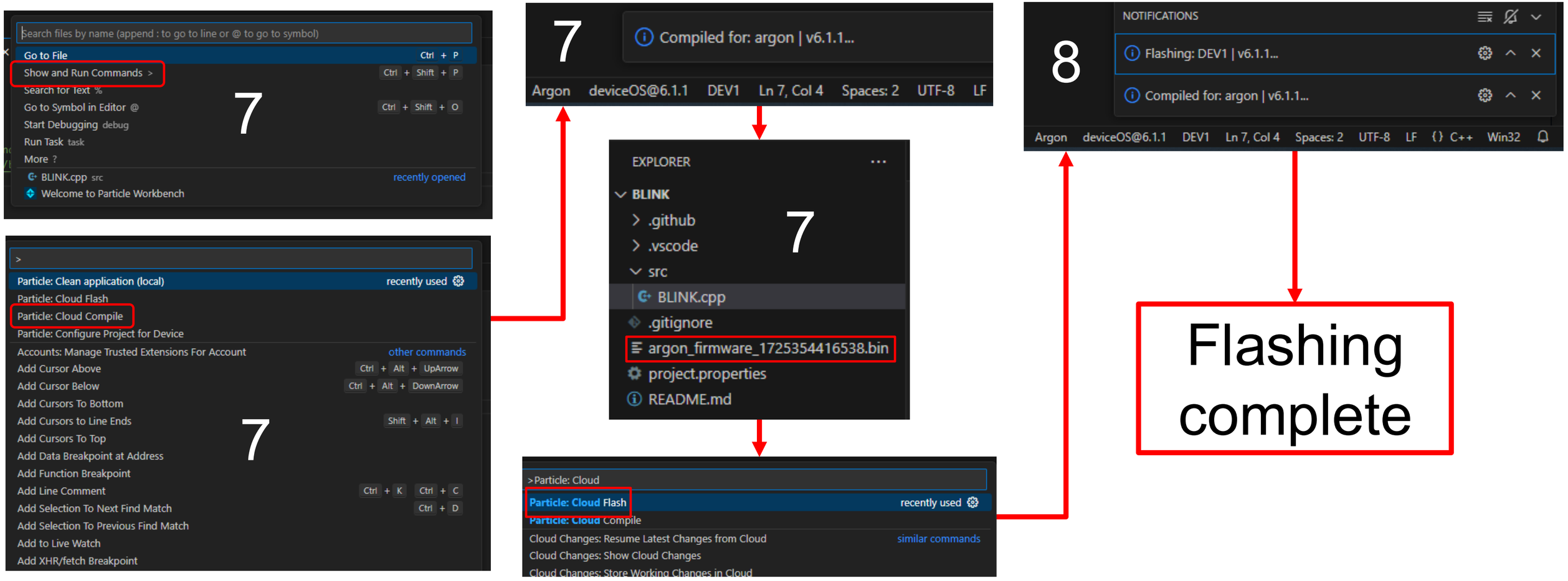






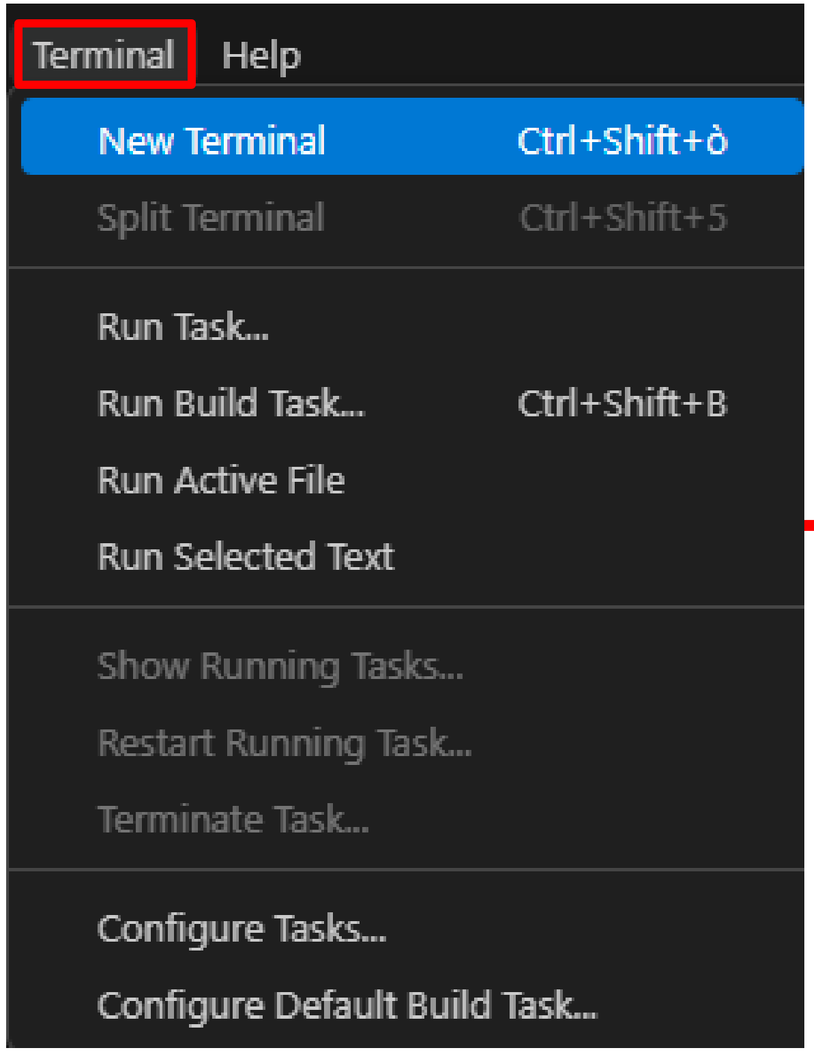
*Make sure that the device is connected from Particle CLI

7. Compile the code
8. Flashing the code

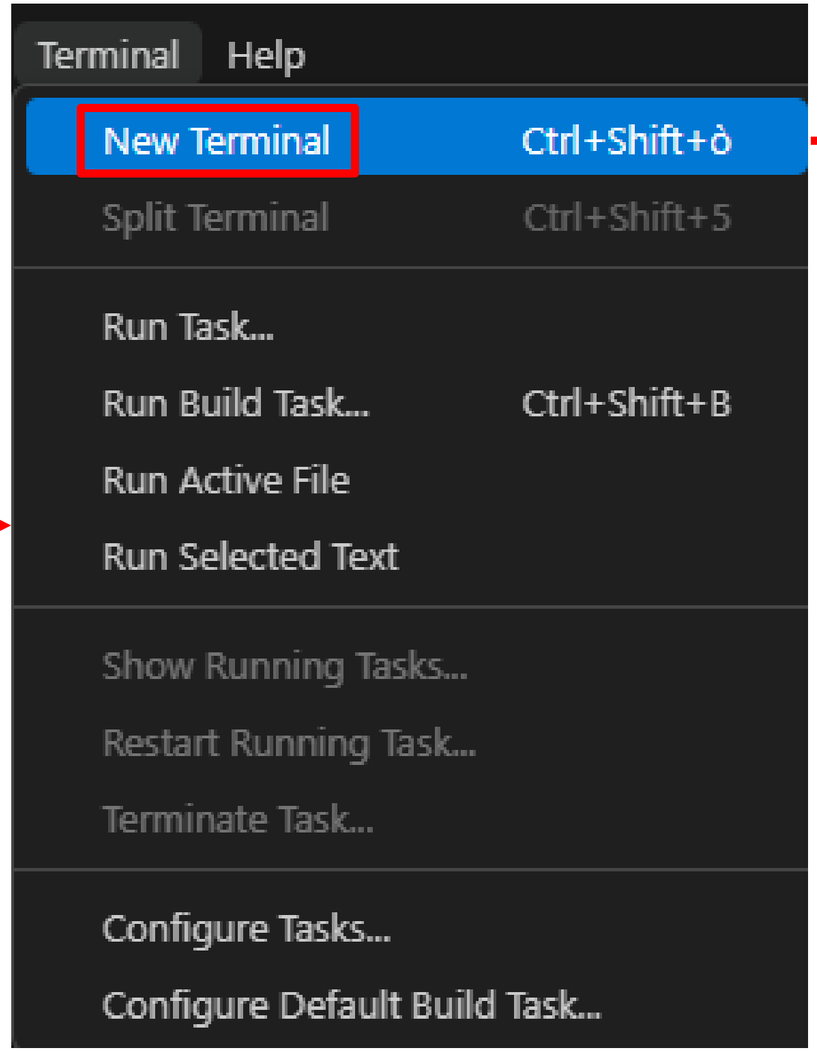


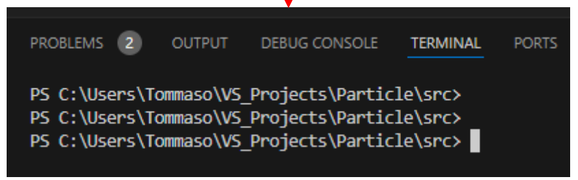
Alternative manner to open a terminal on VS

1



2





Example 1: Compiling and Flashing with CLI

Copy and paste the Code for Blinking the LED: LINK

CLI for only compiling the code and Generating the firmware:

```
particle compile argon E1_Web-Connected-LED_CLI.cpp
```

CLI for flashing the code without generating the firmware:

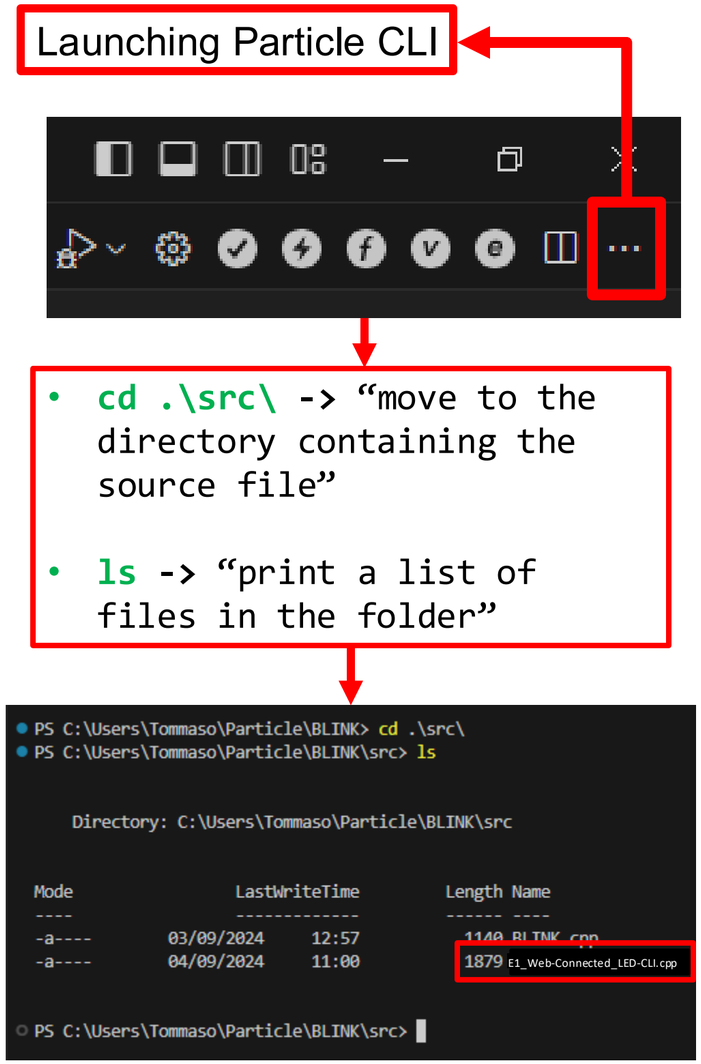
```
particle flash DEV1 E1_Web-Connected-LED_CLI.cpp
```

CLI for blinking the D7 LED for 1000 milliseconds delay:

```
particle call DEV1 BLINK 1000
```

CLI for turning off the D7 LED:

```
particle call DEV1 BLINK OFF
```



No need additional modules on the argon device

Example 1: Function CALL with CLI

Copy and paste the Code for Blinking the LED: LINK

```
int ChangeStatus(String State) { // (State.toInt() != 0);

    if (State == "OFF") { // Turn off the light
        Light_Flag = false;
        return 0;
    }

    // Convert to integer the String
    int newDelay = State.toInt();

    // Check if delay is valid number
    if (newDelay > 0) {
        blinkDelay = newDelay;
        Light_Flag = true;
        return 1;
    } else {
        return -1;
    }

}
```

The function “ChangeStatus” in the code can be called from CLI with an input string to change the blinking delay if D7 LED or turning off the LED.

```
CLI for blinking the D7 LED for 1000 milliseconds delay:
particle call DEV1 BLINK 1000

CLI for turning off the D7 LED:
particle call DEV1 BLINK OFF
```

No need additional modules on the argon device

Example 2: Forcing flash with USB cable and CLI

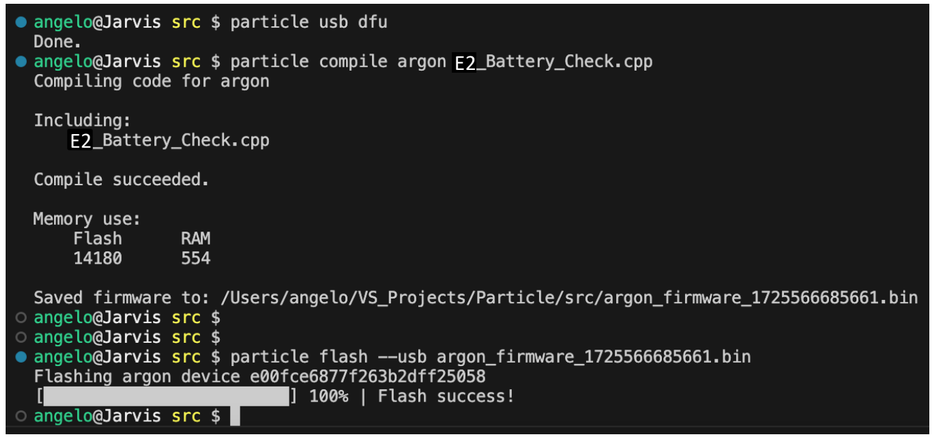
1. Download the code for battery check: LINK
2. Connect the Particle argon to the USB cable
3. Open Particle CLI
4. Follow the steps below:

A. Put the particle Argon in DFU mode (Blinking Yellow)

```
particle usb dfu
```

B. Compile and generate the firmware:

```
particle compile argon E2_Battery_Check.cpp
```



C. Force Flashing the code through usb cable

```
particle flash --usb argon_firmware_XXXXXXXXXXXXX.bin
```

No need additional modules on the argon device

Example 2: GET variables values from CLI

CLI for compiling the code and Generating the firmware (Only for compiling and generating the firmware):

```
particle compile argon E2_Battery_Check.cpp
```

CLI for flashing the firmware:

```
particle flash DEV1 argon_firmware_XXXXXXXXXXXXX.bin
```

CLI for compiling and flashing the code without generating the firmware (Compiling and Flashing directly to the Device):

```
particle flash DEV1 E2_Battery_Check.cpp
```

CLI for getting the Battery Voltage on the screen as a value (Get the battery voltage on screen):

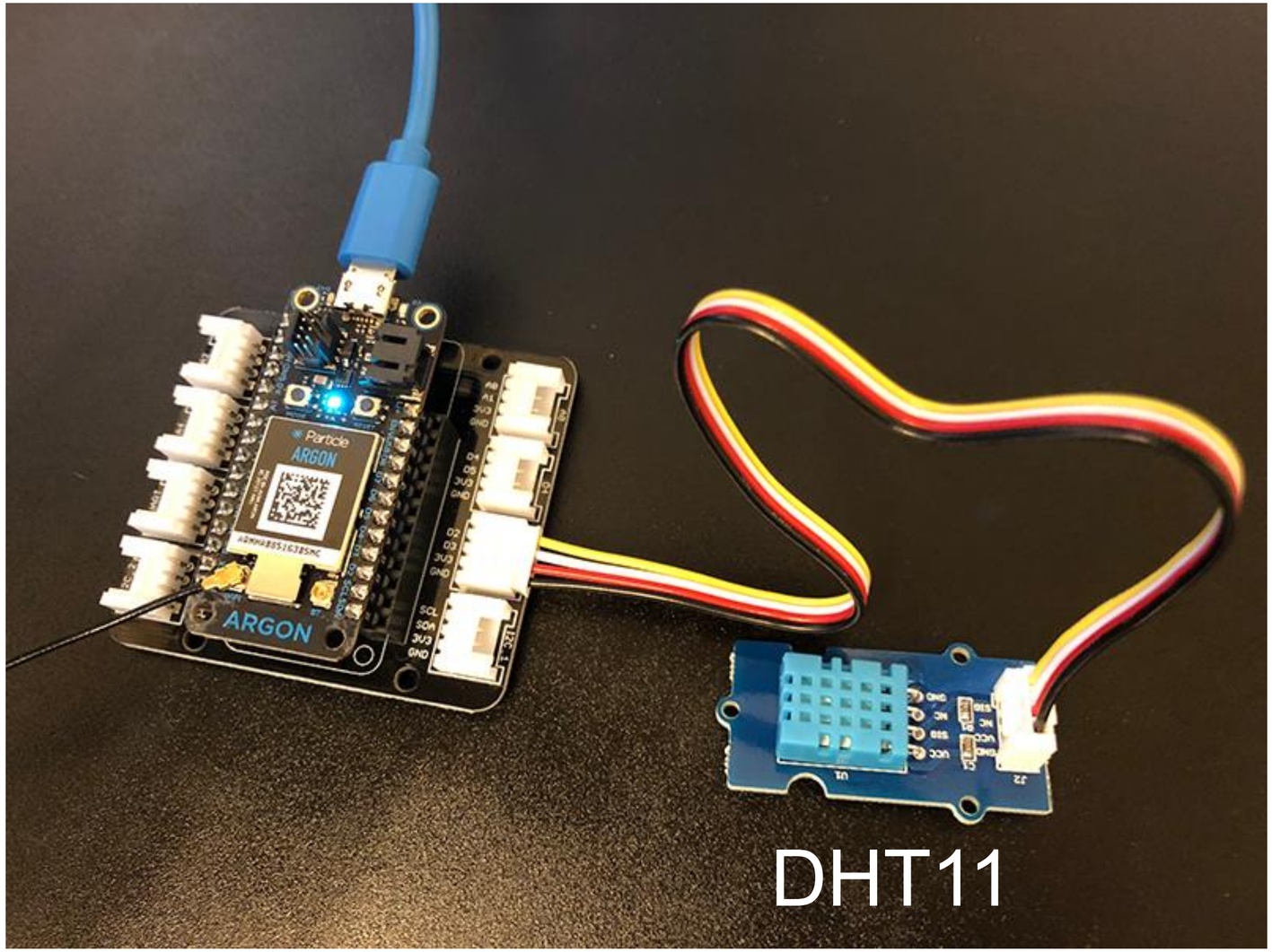
```
particle get DEV1 BatteryVoltage
```

CLI for getting the charging status (true or false) on screen as a string:

```
particle get DEV1 Charging
```

No need additional modules on the argon device

Example 3: Compiling and flashing with Libraries



DHT11 module needed on the argon device in D2 plug on the grove-shield

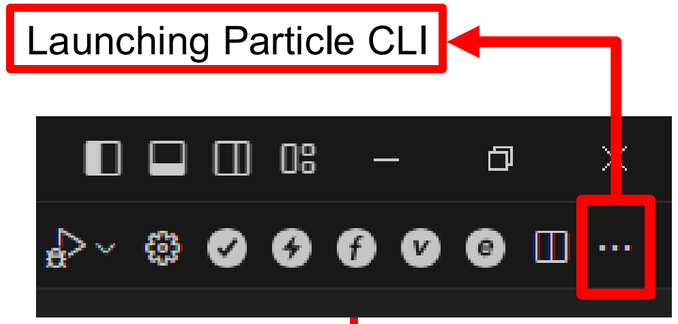
Example 3: Compiling and flashing with Libraries

1. Open Particle CLI
2. Type the following command as example

```
particle library search Adafruit
```

```
    publish  Publish a private library, making it public
    view     View details about a library

  Global Options:
    -v, --verbose  Increases how much logging to display
    -q, --quiet    Decreases how much logging to display

● angelo@Jarvis src $ particle library search Adafruit
  > Found 10 libraries matching Adafruit
  Adafruit_DHT 0.0.4 416832 Spark Core DHT Library based on Adafruit Arduino DHT Library
  Adafruit_Sensor 1.0.2 372693 Required for all Adafruit Unified Sensor based libraries.
  Adafruit_SSD1306 0.0.2 313958 A Port of the ADAFruit SSD1306 libary
  Adafruit_DHT_Particle 0.0.2 198486 Spark Core DHT Library based on Adafruit Arduino DHT Library
  Adafruit_BME280 1.1.5 179702 Adafruit BME280 Library
  Adafruit_GFX_RK 1.11.10 154811 Adafruit GFX graphics core library, this is the 'core' class that
  kas7.
  Adafruit_MCP23017 1.0.2 137224 Adafruit_MCP23017 I2C expander library adapted for Spark
  Adafruit_SSD1306_RK 1.3.3 100252 SSD1306 oled driver library for 'monochrome' 128x64 and 128x32
  Adafruit_mfGFX 1.0.4 95998 Adafruit mfGFX multifont library for Particle Devices
  adafruit-sht31 0.0.7 93004 Adafruit SHT31 port to Particle
○ angelo@Jarvis src $
```

2

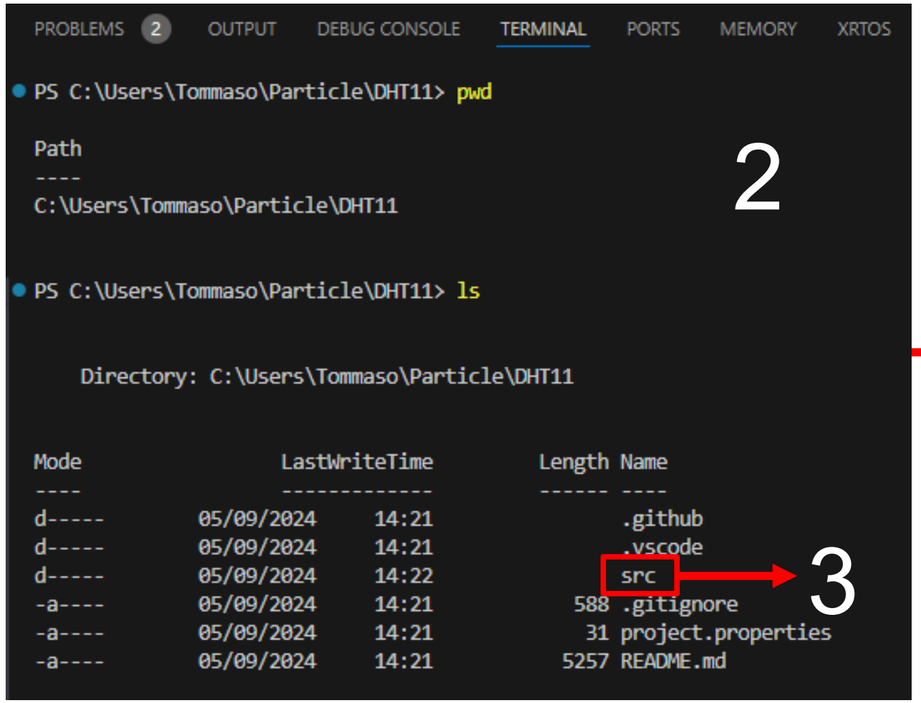


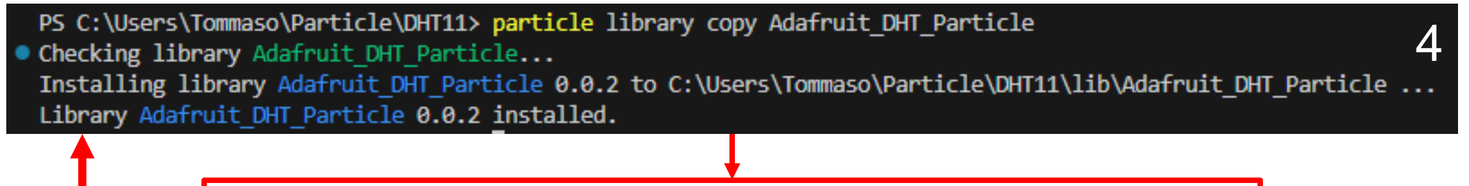
Choose the library for your project

DHT11 module needed on the argon device in D2 plug on the grove-shield

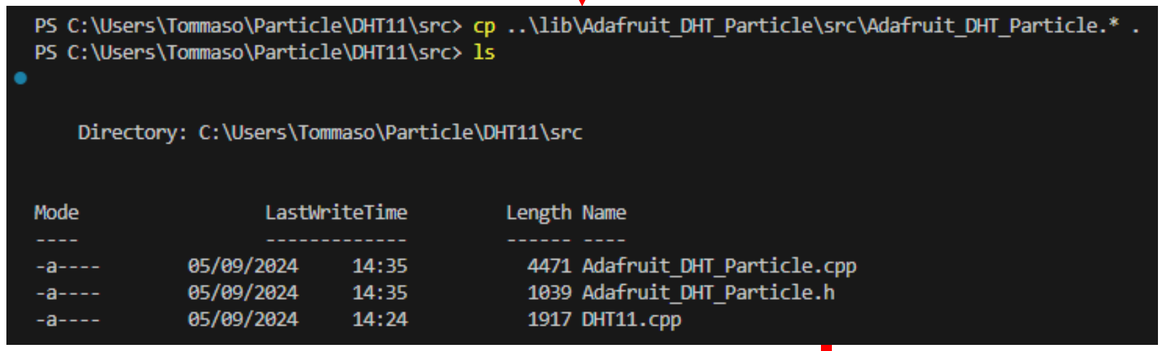
Example 3: Compiling and flashing with Libraries

1. Create a Project in VS named “DHT11”
2. Make sure that you are in the main folder “DHT11”
3. Download the code HERE and move it to the “src” folder. The code requires the *Adafruit_DHT_Particle* library
4. Download the library from the cloud: `particle library copy Adafruit_DHT_Particle`





6. This will create a “lib” folder that contains the header files of the library (*.cpp and *.h)

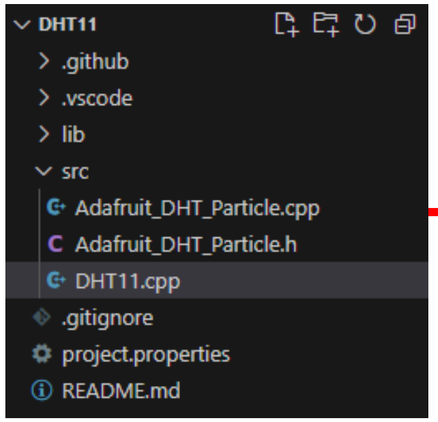


7. Copy the header files into the “src” folder

Example 3: Compiling and flashing with Libraries

Flash the code using the following CLI:

```
particle flash DEV1 DHT11.cpp Adafruit_DHT_Particle.cpp Adafruit_DHT_Particle.h
```




```
PS C:\Users\Tommaso\Particle\DHT11\src> particle flash DEV1 .\DHT11.cpp .\Adafruit_DHT_Particle.cpp .\Adafruit_DHT_Particle.h
Compiling code for DEV1

Including:
    .\DHT11.cpp
    .\Adafruit_DHT_Particle.cpp
    .\Adafruit_DHT_Particle.h

Compile succeeded.

Memory use:
    Flash       RAM
    15520       470

Flashing firmware to your device DEV1
Flash success!
PS C:\Users\Tommaso\Particle\DHT11\src>
```

Flashing complete

Alternative method:

1. Compiling the code and generating the firmware.
2. Flash the firmware.

```
PS C:\Users\Tommaso\Particle\DHT11\src> particle compile argon .\DHT11.cpp .\Adafruit_DHT_Particle.cpp .\Adafruit_DHT_Particle.h
Compiling code for argon

Including:
    .\DHT11.cpp
    .\Adafruit_DHT_Particle.cpp
    .\Adafruit_DHT_Particle.h

Compile succeeded.

Memory use:
    Flash       RAM
    15520       470

Saved firmware to: C:\Users\Tommaso\Particle\DHT11\src\argon_firmware_1725541152503.bin
PS C:\Users\Tommaso\Particle\DHT11\src> particle flash DEV1 .\argon_firmware_1725541152503.bin
Flashing firmware to your device DEV1
Flash success!
PS C:\Users\Tommaso\Particle\DHT11\src>
```

Flashing complete

DHT11 module needed on the argon device in D2 plug on the grove-shield

Example 3: Compiling and flashing with Libraries

CLI for compiling the code and Generating the firmware:

```
particle compile argon DHT11.cpp Adafruit_DHT_Particle.cpp Adafruit_DHT_Particle.h
```

CLI for flashing the firmware:

```
particle flash DEV1 argon_firmware_XXXXXXXXXXXXX.bin
```

CLI for compiling and flashing the code without generating the firmware:

```
particle flash DEV1 DHT11.cpp Adafruit_DHT_Particle.cpp Adafruit_DHT_Particle.h
```

CLI for opening the serial monitor

```
particle serial monitor --follow
```

The serial monitor should print the data from DHT11 module. If the serial monitor is closed or killed, the data are published to the cloud on the specific topic “Readings” (Have a look on the Web-IDE)

DHT11 module needed on the argon device in D2 plug on the grove-shield

Shell Basic Commands (1)

Function	PowerShell Command (Windows Users)	Bash Command (Linux, MacOS Users)
List directory contents	Get-Child Item or ls	ls
Change directory	Set-Location or cd	cd
Print working directory	Get-Location	pwd
Copy files	Copy-Item	cp
Move files	Move-Item	mv
Delete files	Remove-Item	rm
Create a directory	New-Item -ItemType Directory (mkdir)	mkdir
Remove a directory	Remove-Item –Recurse (del)	rm -r
Rename a file	Rename-Item	mv
Display file content	Get-Content	cat
Search inside files	Select-String	grep
Find command history	Get-History	history
Clear screen	Clear-Host or cls	clear
Run a script	.\script.ps1	./script.sh
Show process list	Get-Process	ps
Kill a process	Stop-Process	kill
Download a file	Invoke-WebRequest	wget or curl
Display system information	Get-ComputerInfo	uname -a
Check network configuration	Get-NetIPConfiguration	ifconfig or ip a
Display disk usage	Get-PSDrive	df
Environment variables	$env:VARIABLE	echo $VARIABLE
Set environment variable	$env:VARIABLE = "value"	export VARIABLE=value

Shell Basic Commands (2)

- `cd ~/Desktop` -- (change the working directory to the Desktop from the home directory “~”)
- `mkdir footage` -- (create a folder named “footage”)
- `pwd` -- (print on screen the current directory)
- `ls` -- (print on screen the list of file and folders inside the working directory)
- `ipconfig` -- (print the available connection interfaces and the ipaddress assigned to them – Windows users)
- `ifconfig` -- (print the available connection interfaces and the ipaddress assigned to them – Linux or MacOS users)
- `netstat –an` -- (print the status of current inbound and outbound connection on your PC)
- `env` -- (Provide a list of enviroment variables)
- `chmod` -- (change the file permissions: Linux or MacOS users)
- `ps` -- (print the list of all the processes)

Shell Basic Operations

Operation	PowerShell Command	Bash Command
For loop	for ($i = 0; $i -lt 5; $i++) { Write-Output $i }	for i in {0..4}; do echo $i; done
ForEach loop	foreach ($item in $array) { Write-Output $item }	for item in "${array[@]}"; do echo $item; done
If statement	if ($x -gt 10) { Write-Output "Greater" }	if [$x -gt 10]; then echo "Greater"; fi
Else statement	if ($x -gt 10) { Write-Output "Greater" } else { Write-Output "Smaller" }	if [$x -gt 10]; then echo "Greater"; else echo "Smaller"; fi
ElseIf statement	if ($x -gt 10) { Write-Output "Greater" } elseif ($x -eq 10) { Write-Output "Equal" } else { Write-Output "Smaller" }	if [$x -gt 10]; then echo "Greater"; elif [$x -eq 10]; then echo "Equal"; else echo "Smaller"; fi
While loop	while ($i -lt 5) { Write-Output $i; $i++ }	while [$i -lt 5]; do echo $i; ((i++)); done
Do-While loop	do { Write-Output $i; $i++ } while ($i -lt 5)	i=0; until [$i -ge 5]; do echo $i; ((i++)); done
Switch statement	switch ($x) { 1 { Write-Output "One" } 2 { Write-Output "Two" } default { Write-Output "Other" } }	case $x in 1) echo "One" ;; 2) echo "Two" ;; *) echo "Other" ;; esac

Example 4: Script for monitoring the connected devices

1. Connect 2 or more devices to the network
2. Download the script for all users: LINK
3. Open the PowerShell (Windows) or the Terminal (Linux, MacOS)

```
Only for Windows users allow execute script permission:
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy Unrestricted

To execute the script type the following command:
.\Connection_Monitoring.ps1
```

```
For Linux/MacOS users, firstly you need to give the execution permissions:
chmod +x Connection_Monitoring.sh

To execute the script type the following command:
./Connection_Monitoring.sh
```

- No need additional modules on the argon device
- At least 1 device must be connected

Example 4: Script for monitoring the connected devices

1. Connect a 2 or more devices to the network
2. Download the script for all users: LINK
3. Open the PowerShell (Windows) or the Terminal (Linux, MacOS)

The script should print on screen the list of the connected devices. If the user tries to shutdown one device, this should appear real-time on the screen as “disconnected device”. It takes roughly 2 minutes because it has to update from the cloud and not locally.

- No need additional modules on the argon device
- At least 1 device must be connected

Example 5: Script for multiple code flashing

Grove-ChainableLED

- Grove-ChainableLED module needed on the argon device for D4, D5 plug on the grove-shield
- Need more than 1 device connected

Example 5: Script for multiple code flashing

Download the script (Flash_Full_Devices.ps1) for flashing the code: LINK

```
$FILE_TEMP = "Subscribe_Publish.temp"

$CODE = "Subscribe_Publish.cpp"
$HEAD1 = "Grove_ChainableLED.cpp"
$HEAD2 = "Grove_ChainableLED.h"

$Devices_SUB = @("DEV1", "DEV2", "DEV3", "DEV4", "DEV5", "DEV6") # Sending the message (You can change it based on the online devices)
$Devices_PUB = @("DEV2", "DEV3", "DEV4", "DEV5", "DEV6", "DEV1") # Receiving the message from "Devices_SUB"

# Get online devices list and store it in a variable
$Device_List = particle list | Select-String "(Argon)" | Select-String "online" | ForEach-Object { $_ -replace "\s+", " " -split " " }

# Loop through each line of the command output
foreach ($Line in $Device_List) {
    # Loop through each device name
    foreach ($Device in $Devices_SUB) {
        # Check if the line matches the device name
        if ($Line -match $Device) {
            # Get index of online devices
            $Index = -1
            foreach ($Dev_SUB in $Devices_SUB) {$Index += 1; if ($Dev_SUB -match $Device) {break}}
            # Acquire content of the template file (*.temp) in the variable $FILE
            $FILE = Get-Content -Path $FILE_TEMP -Raw
            # Substitute the keywords of the *.temp file with the device name
            $FILE = $FILE -replace "XXX", $Device
            $FILE = $FILE -replace "YYY", $Devices_PUB[$Index]
            # Redirect output of the modified *.temp file into *.cpp file
            Write-Output $FILE > $CODE
            # Flashing the corresponding modified code
            Write-Output "Flashing: $Device"
            particle flash $Device $CODE $HEAD1 $HEAD2
        }
    }
}
```

- Grove-ChainableLED module needed on the argon device for D4, D5 plug on the grove-shield
- Need more than 1 device connected

Example 5: Code to execute the LED fire

2. Download the rest of the code in the following LINK, and save it in the same folder.

```
void publishCallback(const char *event, const char *data) {
        String deviceID = System.deviceID();
        Log.info("event=%s data=%s device=%s", event, (data ? data : "NULL"), deviceID.c_str());
        if (String(event) == "CAPCOM" && String(data) == "ON") {
                Trigger = true;
        } else if (String(event) == "CAPCOM" && String(data) == "OFF") {
                Trigger = false;
        }
        if (String(event) == "Devices" && String(data) == "XXX" && Trigger) {TriggerRemote();};
}
```

The function “publishCallback” in the template file is executed when a publish event from cloud is executed. The string “XXX” is substituted with the device name

```
// Function to trigger remote devices to blink the LED and send message to the consecutive device
void TriggerRemote() {
        leds.setColorRGB(0, 255, 255, 255);
        delay(500);
        leds.setColorRGB(0, 0, 0, 0);
        Particle.publish("Devices", "YYY");
}
```

The function “TriggerRemote” in the template file is executed when the “publishCallback” function is called due to a publish event. The function fires the LED for 500 milliseconds and publish an event on “Devices” topic with a device name as message. The string “YYY” is the message that must substituted with the device name to be queried for the next LED fire.

- Grove-ChainableLED module needed on the argon device for D4, D5 plug on the grove-shield
- Need more than 1 device connected

Example 5: Execution CLI sequence

3. Execute the following script:

```
./Flash_Full_Devices.ps1 (flash the generated code to each specific device if they are online)
```

4. Type the following commands on CLI:

```
particle publish CAPCOM ON (Activate Triggering)
particle publish Devices DEV1 (Start Trigger)
particle publish CAPCOM OFF (Stop Trigger)
```

- Grove-ChainableLED module needed on the argon device for D4, D5 plug on the grove-shield
- Need more than 1 device connected

Example 5: Lighting sequence

- particle publish CAPCOM ON (Activate the lighting sequence between the devices. Set the “Trigger” boolean to true)
- particle publish Devices DEV1 (Start the lighting sequence between the assigned devices. One device turns-on the light for 500 millisecond and send the signal to another device. The sequence is defined in the “Flash_Full_Devices.ps1” script)
- particle publish CAPCOM OFF (Kills the lighting sequence. Set the “Trigger” boolean to false)

- Grove-ChainableLED module needed on the argon device for D4, D5 plug on the grove-shield
- Need more than 1 device connected

gastone.ciuti@santannapisa.it